Chapter 6. Working with Sets

Although you can interact with the data in a database one row at a time, relational databases are really all about sets. This chapter explores how you can combine multiple result sets using various set operators. After a quick overview of set theory, I'll demonstrate how to use the set operators `union`, `intersect`, and `except` to blend multiple data sets together.

Set Theory Primer

In many parts of the world, basic set theory is included in elementary-level math curriculums. Perhaps you recall looking at something like what is shown in Figure 6-1.

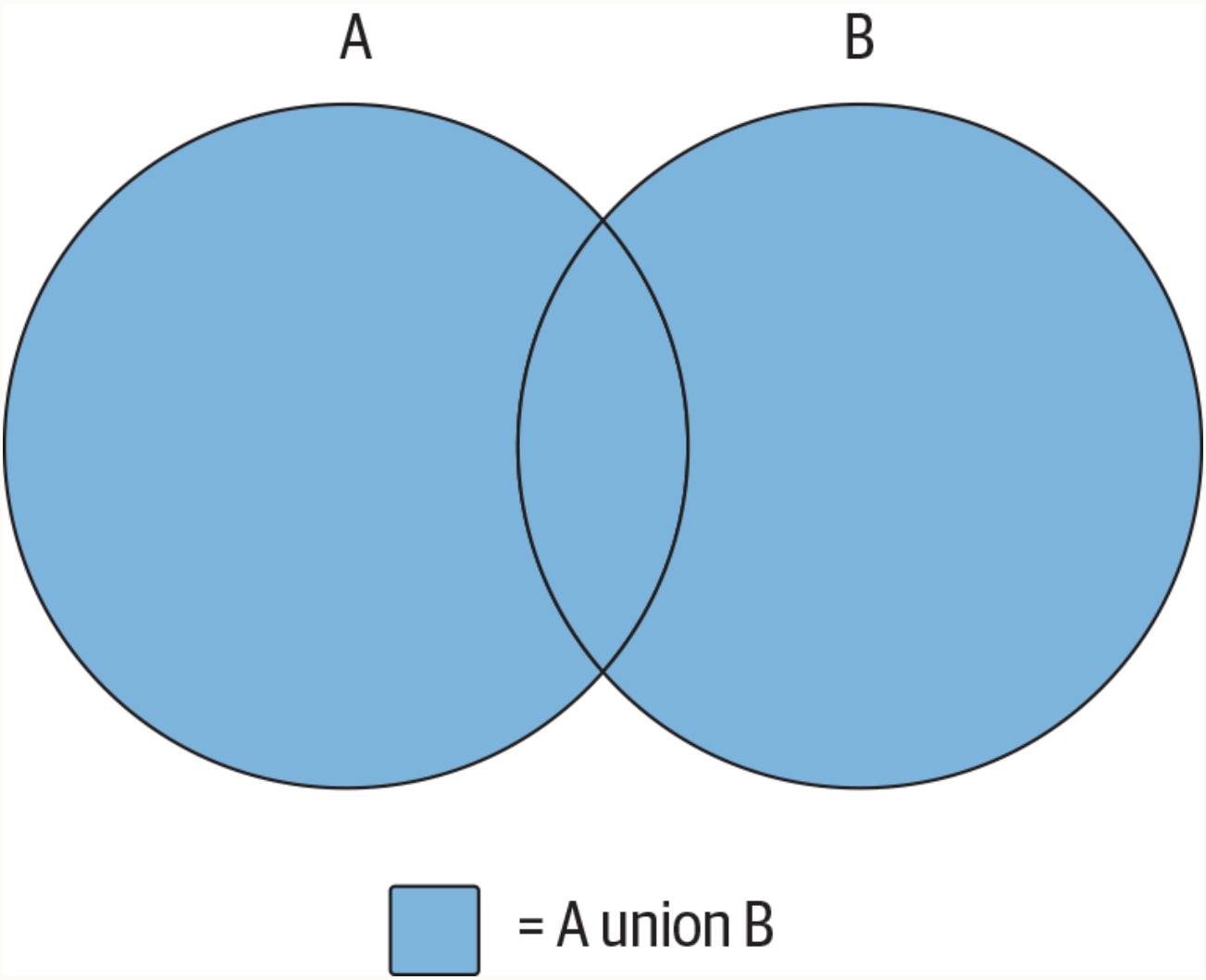


Figure 6-1. The union operation

The shaded area in Figure 6-1 represents the *union* of sets A and B, which is the combination of the two sets (with any overlapping regions included only once). Is this starting to look familiar? If so, then you'll finally get a chance to put that knowledge to use; if not, don't worry, because it's easy to visualize using a couple of diagrams.

Using circles to represent two data sets (A and B), imagine a subset of data that is common to both sets; this common data is represented by the overlapping area shown in Figure 6-1. Since set theory is rather uninteresting without an overlap between data sets, I use the same diagram to illustrate each set operation. There is another set operation that is concerned *only* with the overlap between two data sets; this operation is known as the *intersection* and is demonstrated in Figure 6-2.

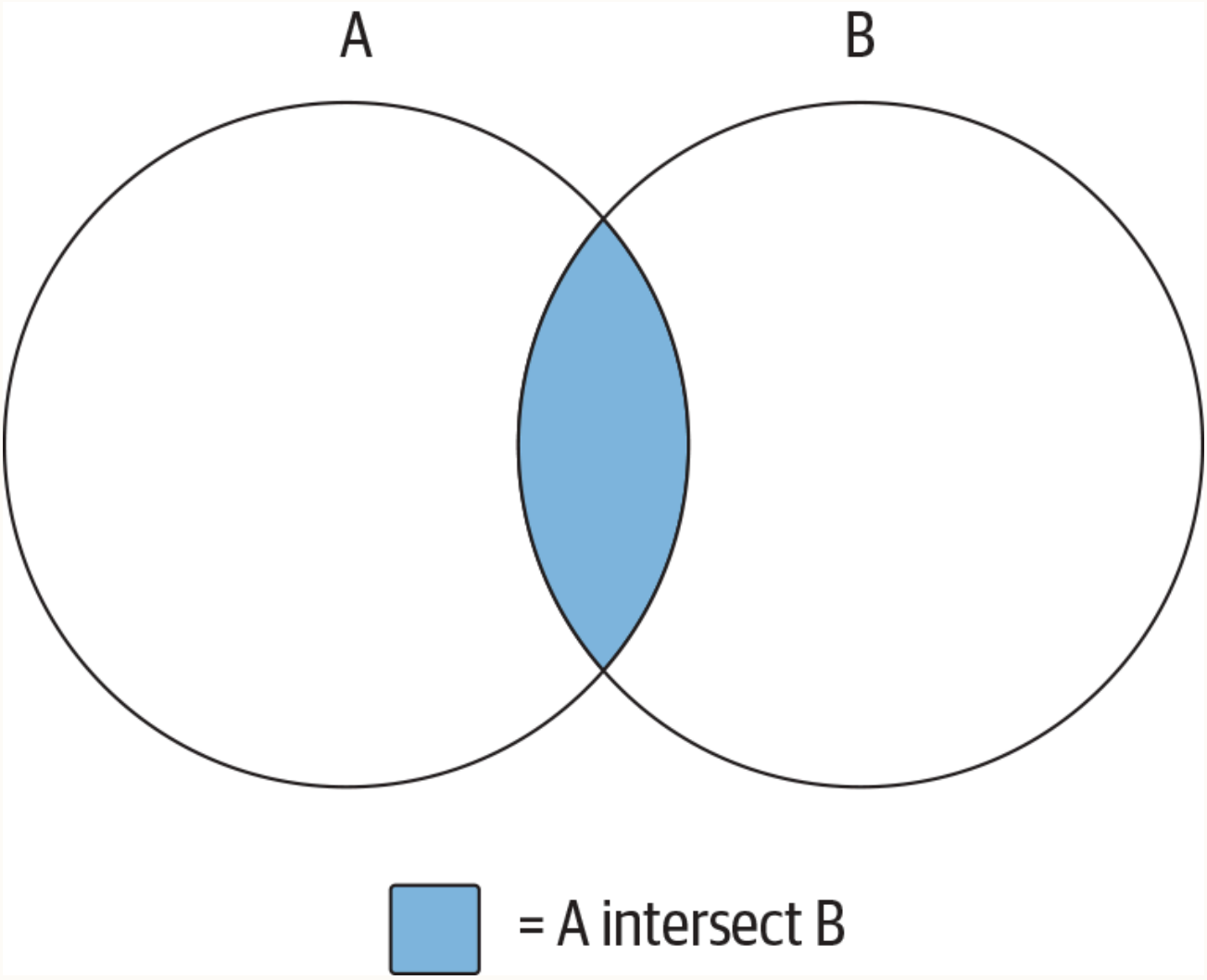


Figure 6-2. The intersection operation

The data set generated by the intersection of sets A and B is just the area of overlap between the two sets. If the two sets have no overlap, then the intersection operation yields the empty set.

The third and final set operation, which is demonstrated in Figure 6-3, is known as the *except* operation.

Figure 6-3 shows the results of `A except B`, which is the whole of set A minus any overlap with set B. If the two sets have no overlap, then the operation `A except B` yields the whole of set A.

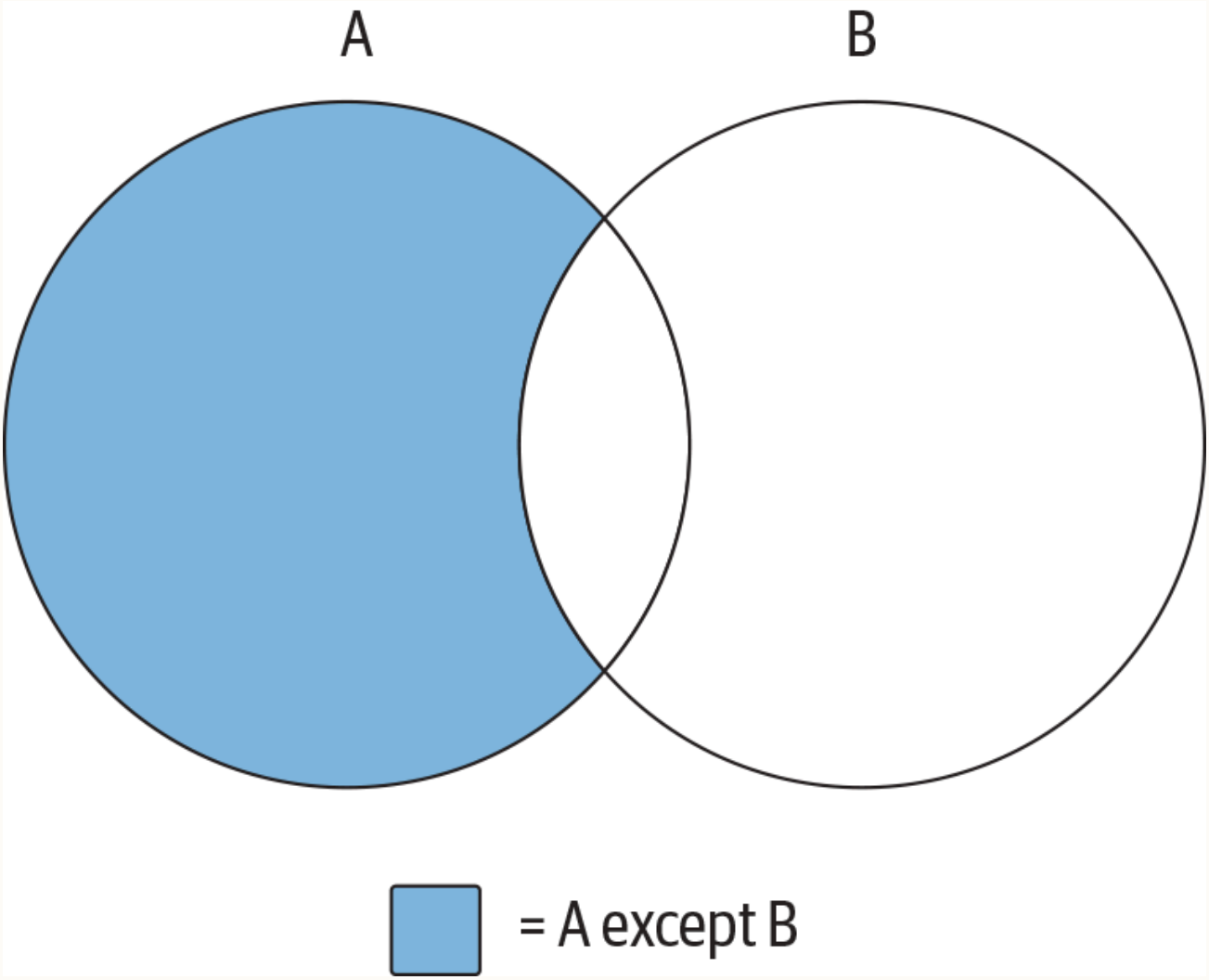


Figure 6-3. The except operation

Using these three operations, or by combining different operations together, you can generate whatever results you need. For example, imagine that you want to build a set demonstrated by Figure 6-4.

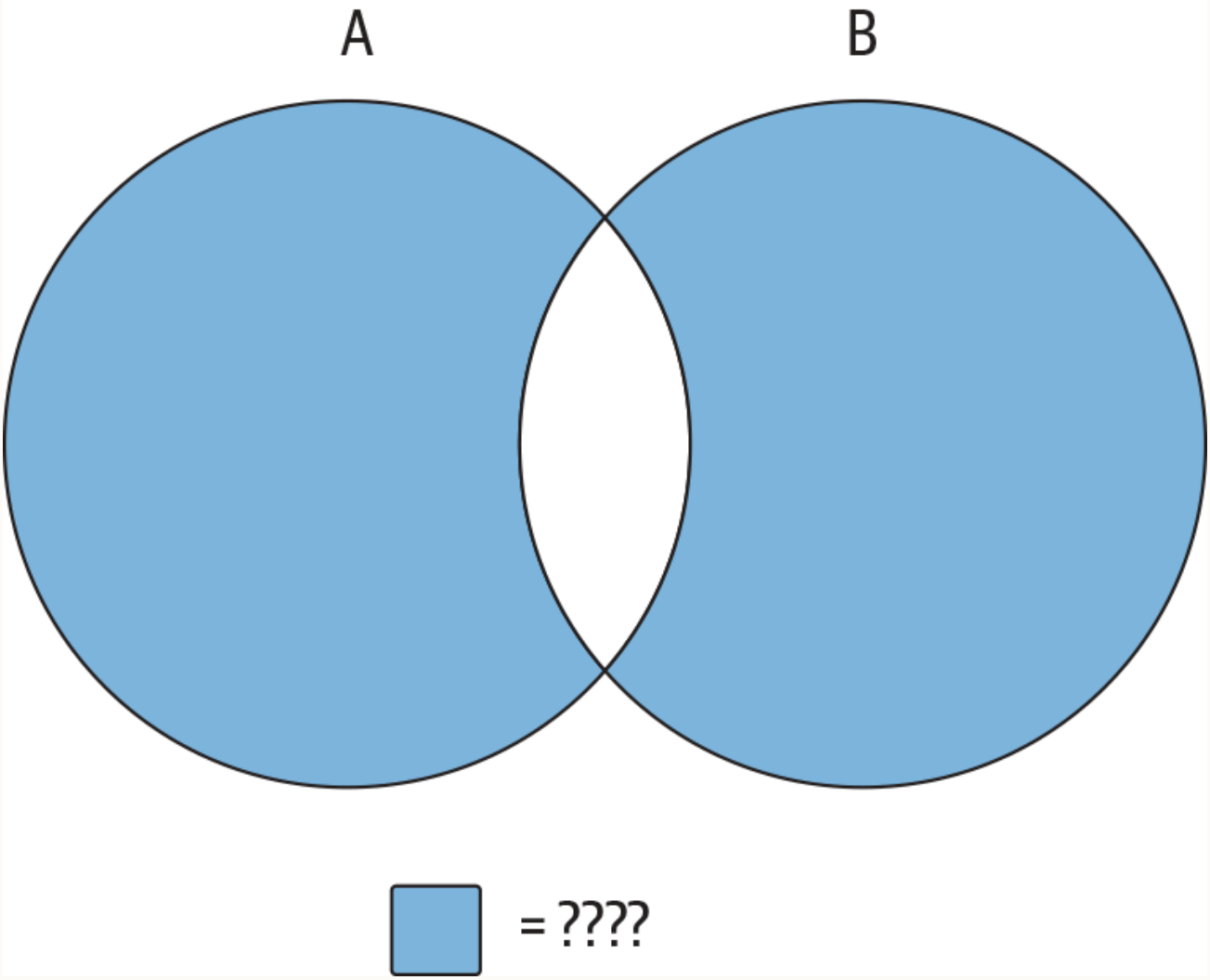


Figure 6-4. Mystery data set

The data set you are looking for includes all of sets A and B *without* the overlapping region. You can't achieve this outcome with just one of the three operations shown earlier; instead, you will need to first build a data set that encompasses all of sets A and B, and then utilize a second operation to remove the overlapping region. If the combined set is described as `A union B`, and the overlapping region is described as `A intersect B`, then the operation needed to generate the data set represented by Figure 6-4 would look as follows:

```
(A union B) except (A intersect B)
```

Of course, there are often multiple ways to achieve the same results; you could reach a similar outcome using the following operation:

```
(A except B) union (B except A)
```

While these concepts are fairly easy to understand using diagrams, the next sections show you how these concepts are applied to a relational database using the SQL set

operators.

Set Theory in Practice

The circles used in the previous section's diagrams to represent data sets don't convey anything about what the data sets comprise. When dealing with actual data, however, there is a need to describe the composition of the data sets involved if they are to be combined. Imagine, for example, what would happen if you tried to generate the union of the `customer` table and the `city` table, whose definitions are as follows:

```
mysql> desc customer;
+-------------+----------------------+------+-----+-------------------+
| Field       | Type                 | Null | Key | Default           |
+-------------+----------------------+------+-----+-------------------+
| customer_id | smallint(5) unsigned | NO   | PRI | NULL              |
| store_id    | tinyint(3) unsigned  | NO   | MUL | NULL              |
| first_name  | varchar(45)          | NO   |     | NULL              |
| last_name   | varchar(45)          | NO   | MUL | NULL              |
| email       | varchar(50)          | YES  |     | NULL              |
| address_id  | smallint(5) unsigned | NO   | MUL | NULL              |
| active      | tinyint(1)           | NO   |     | 1                 |
| create_date | datetime             | NO   |     | NULL              |
| last_update | timestamp            | YES  |     | CURRENT_TIMESTAMP |
+-------------+----------------------+------+-----+-------------------+

mysql> desc city;
+-------------+----------------------+------+-----+-------------------+
| Field       | Type                 | Null | Key | Default           |
+-------------+----------------------+------+-----+-------------------+
| city_id     | smallint(5) unsigned | NO   | PRI | NULL              |
| city        | varchar(50)          | NO   |     | NULL              |
| country_id  | smallint(5) unsigned | NO   | MUL | NULL              |
| last_update | timestamp            | NO   |     | CURRENT_TIMESTAMP |
+-------------+----------------------+------+-----+-------------------+
```

When combined, the first column in the result set would include both the `customer.customer_id` and `city.city_id` columns, the second column would be the combination of the `customer.store_id` and `city.city` columns, and so forth. While some column pairs are easy to combine (e.g., two numeric columns), it is unclear how other column pairs should be combined, such as a numeric column with a string column or a string column with a date column. Additionally, the fifth through ninth columns of the com-

bined tables would include data from only the `customer` table's fifth through ninth columns, since the `city` table has only four columns. Clearly, there needs to be some commonality between two data sets that you wish to combine.

Therefore, when performing set operations on two data sets, the following guidelines must apply:

- Both data sets must have the same number of columns.
- The data types of each column across the two data sets must be the same (or the server must be able to convert one to the other).

With these rules in place, it is easier to envision what "overlapping data" means in practice; each column pair from the two sets being combined must contain the same string, number, or date for rows in the two tables to be considered the same.

You perform a set operation by placing a *set operator* between two `select` statements, as demonstrated by the following:

```
mysql> SELECT 1 num, 'abc' str
    -> UNION
    -> SELECT 9 num, 'xyz' str;
+-----+-----+
| num | str |
+-----+-----+
|   1 | abc |
|   9 | xyz |
+-----+-----+
2 rows in set (0.02 sec)
```

Each of the individual queries yields a data set consisting of a single row having a numeric column and a string column. The set operator, which in this case is `union`, tells the database server to combine all rows from the two sets. Thus, the final set includes two rows of two columns. This query is known as a *compound query* because it comprises multiple, otherwise-independent queries. As you will see later, compound queries may include *more* than two queries if multiple set operations are needed to attain the final results.

Set Operators

The SQL language includes three set operators that allow you to perform each of the various set operations described earlier in the chapter. Additionally, each set operator has two flavors, one that includes duplicates and another that removes duplicates (but not necessarily *all* of the duplicates). The following subsections define each operator and demonstrate how they are used.

The union Operator

The `union` and `union all` operators allow you to combine multiple data sets. The difference between the two is that `union` sorts the combined set and removes duplicates, whereas `union all` does not. With `union all`, the number of rows in the final data set will always equal the sum of the number of rows in the sets being combined. This operation is the simplest set operation to perform (from the server's point of view), since there is no need for the server to check for overlapping data. The following example demonstrates how you can use the `union all` operator to generate a set of first and last names from multiple tables:

```
mysql> SELECT 'CUST' typ, c.first_name, c.last_name
    -> FROM customer c
    -> UNION ALL
    -> SELECT 'ACTR' typ, a.first_name, a.last_name
    -> FROM actor a;
+------+------------+-------------+
| typ  | first_name | last_name   |
+------+------------+-------------+
| CUST | MARY       | SMITH       |
| CUST | PATRICIA   | JOHNSON     |
| CUST | LINDA      | WILLIAMS    |
| CUST | BARBARA    | JONES       |
| CUST | ELIZABETH  | BROWN       |
| CUST | JENNIFER   | DAVIS       |
| CUST | MARIA      | MILLER      |
| CUST | SUSAN      | WILSON      |
| CUST | MARGARET   | MOORE       |
| CUST | DOROTHY    | TAYLOR      |
| CUST | LISA       | ANDERSON    |
| CUST | NANCY      | THOMAS      |
| CUST | KAREN      | JACKSON     |
...
| ACTR | BURT       | TEMPLE      |
```

```
| ACTR | MERYL     | ALLEN       |
| ACTR | JAYNE     | SILVERSTONE |
| ACTR | BELA      | WALKEN      |
| ACTR | REESE     | WEST        |
| ACTR | MARY      | KEITEL      |
| ACTR | JULIA     | FAWCETT     |
| ACTR | THORA     | TEMPLE      |
+------+-----------+-------------+
799 rows in set (0.00 sec)
```

The query returns 799 names, with 599 rows coming from the `customer` table and the other 200 coming from the `actor` table. The first column, which has the alias `typ`, is not necessary, but was added to show the source of each name returned by the query.

Just to drive home the point that the `union all` operator doesn't remove duplicates, here's another version of the previous example, but with two identical queries against the `actor` table:

```
mysql> SELECT 'ACTR' typ, a.first_name, a.last_name
    -> FROM actor a
    -> UNION ALL
    -> SELECT 'ACTR' typ, a.first_name, a.last_name
    -> FROM actor a;
+------+-------------+--------------+
| typ  | first_name  | last_name    |
+------+-------------+--------------+
| ACTR | PENELOPE    | GUINESS      |
| ACTR | NICK        | WAHLBERG     |
| ACTR | ED          | CHASE        |
| ACTR | JENNIFER    | DAVIS        |
| ACTR | JOHNNY      | LOLLOBRIGIDA |
| ACTR | BETTE       | NICHOLSON    |
| ACTR | GRACE       | MOSTEL       |
...
| ACTR | BURT        | TEMPLE       |
| ACTR | MERYL       | ALLEN        |
| ACTR | JAYNE       | SILVERSTONE  |
| ACTR | BELA        | WALKEN       |
| ACTR | REESE       | WEST         |
| ACTR | MARY        | KEITEL       |
| ACTR | JULIA       | FAWCETT      |
| ACTR | THORA       | TEMPLE       |
```

```
+------+------------+--------------+
400 rows in set (0.00 sec)
```

As you can see by the results, the 200 rows from the `actor` table are included twice, for a total of 400 rows.

While you are unlikely to repeat the same query twice in a compound query, here is another compound query that returns duplicate data:

```
mysql> SELECT c.first_name, c.last_name
    -> FROM customer c
    -> WHERE c.first_name LIKE 'J%' AND c.last_name LIKE 'D%'
    -> UNION ALL
    -> SELECT a.first_name, a.last_name
    -> FROM actor a
    -> WHERE a.first_name LIKE 'J%' AND a.last_name LIKE 'D%';
+------------+-----------+
| first_name | last_name |
+------------+-----------+
| JENNIFER   | DAVIS     |
| JENNIFER   | DAVIS     |
| JUDY       | DEAN      |
| JODIE      | DEGENERES |
| JULIANNE   | DENCH     |
+------------+-----------+
5 rows in set (0.00 sec)
```

Both queries return the names of people having the initials JD. Of the five rows in the result set, one of them is a duplicate (Jennifer Davis). If you would like your combined table to *exclude* duplicate rows, you need to use the `union` operator instead of `union all`:

```
mysql> SELECT c.first_name, c.last_name
    -> FROM customer c
    -> WHERE c.first_name LIKE 'J%' AND c.last_name LIKE 'D%'
    -> UNION
    -> SELECT a.first_name, a.last_name
    -> FROM actor a
    -> WHERE a.first_name LIKE 'J%' AND a.last_name LIKE 'D%';
+------------+-----------+
| first_name | last_name |
+------------+-----------+
| JENNIFER   | DAVIS     |
```

```
| JUDY       | DEAN      |
| JODIE      | DEGENERES |
| JULIANNE   | DENCH     |
+------------+-----------+
4 rows in set (0.00 sec)
```

For this version of the query, only the four distinct names are included in the result set, rather than the five rows returned when using `union all`.

The intersect Operator

The ANSI SQL specification includes the `intersect` operator for performing intersections. Unfortunately, version 8.0 of MySQL does not implement the `intersect` operator. If you are using Oracle or SQL Server 2008, you will be able to use `intersect`; since I am using MySQL for all examples in this book, however, the result sets for the example queries in this section are fabricated and cannot be executed with any versions up to and including version 8.0. I also refrain from showing the MySQL prompt (`mysql>`), since the statements are not being executed by the MySQL server.

If the two queries in a compound query return nonoverlapping data sets, then the intersection will be an empty set. Consider the following query:

```
SELECT c.first_name, c.last_name
FROM customer c
WHERE c.first_name LIKE 'D%' AND c.last_name LIKE 'T%'
INTERSECT
SELECT a.first_name, a.last_name
FROM actor a
WHERE a.first_name LIKE 'D%' AND a.last_name LIKE 'T%';
Empty set (0.04 sec)
```

While there are both actors and customers having the initials DT, these sets are completely nonoverlapping, so the intersection of the two sets yields the empty set. If we switch back to the initials JD, however, the intersection will yield a single row:

```
SELECT c.first_name, c.last_name
FROM customer c
WHERE c.first_name LIKE 'J%' AND c.last_name LIKE 'D%'
INTERSECT
SELECT a.first_name, a.last_name
FROM actor a
```

```
WHERE a.first_name LIKE 'J%' AND a.last_name LIKE 'D%';
+------------+-----------+
| first_name | last_name |
+------------+-----------+
| JENNIFER   | DAVIS     |
+------------+-----------+
1 row in set (0.00 sec)
```

The intersection of these two queries yields Jennifer Davis, which is the only name found in both queries' result sets.

Along with the `intersect` operator, which removes any duplicate rows found in the overlapping region, the ANSI SQL specification calls for an `intersect all` operator, which does not remove duplicates. The only database server that currently implements the `intersect all` operator is IBM's DB2 Universal Server.

The except Operator

The ANSI SQL specification includes the `except` operator for performing the except operation. Once again, unfortunately, version 8.0 of MySQL does not implement the `except` operator, so the same rules apply for this section as for the previous section.

NOTE

If you are using Oracle Database, you will need to use the non-ANSI-compliant `minus` operator instead.

The `except` operator returns the first result set minus any overlap with the second result set. Here's the example from the previous section, but using `except` instead of `intersect`, and with the order of the queries reversed:

```
SELECT a.first_name, a.last_name
FROM actor a
WHERE a.first_name LIKE 'J%' AND a.last_name LIKE 'D%'
EXCEPT
SELECT c.first_name, c.last_name
FROM customer c
WHERE c.first_name LIKE 'J%' AND c.last_name LIKE 'D%';
+------------+-----------+
| first_name | last_name |
```

```
+-----------+-----------+
| JUDY      | DEAN      |
| JODIE     | DEGENERES |
| JULIANNE  | DENCH     |
+-----------+-----------+
3 rows in set (0.00 sec)
```

In this version of the query, the result set consists of the three rows from the first query minus Jennifer Davis, who is found in the result sets from both queries. There is also an `except all` operator specified in the ANSI SQL specification, but once again, only IBM's DB2 Universal Server has implemented the `except all` operator.

The `except all` operator is a bit tricky, so here is an example that demonstrates how duplicate data is handled. Let's say you have two data sets that look like the following:

Set A

```
+----------+
| actor_id |
+----------+
|       10 |
|       11 |
|       12 |
|       10 |
|       10 |
+----------+
```

Set B

```
+----------+
| actor_id |
+----------+
|       10 |
|       10 |
+----------+
```

The operation `A except B` yields the following:

```
+----------+
| actor_id |
+----------+
```

```
|      11 |
|      12 |
+----------+
```

If you change the operation to `A except all B`, you will see the following:

```
+----------+
| actor_id |
+----------+
|      10 |
|      11 |
|      12 |
+----------+
```

Therefore, the difference between the two operations is that `except` removes all occurrences of duplicate data from set A, whereas `except all` removes only one occurrence of duplicate data from set A for every occurrence in set B.

Set Operation Rules

The following sections outline some rules that you must follow when working with compound queries.

Sorting Compound Query Results

If you want the results of your compound query to be sorted, you can add an `order by` clause after the last query. When specifying column names in the `order by` clause, you will need to choose from the column names in the first query of the compound query. Frequently, the column names are the same for both queries in a compound query, but this does not need to be the case, as demonstrated by the following:

```
mysql> SELECT a.first_name fname, a.last_name lname
    -> FROM actor a
    -> WHERE a.first_name LIKE 'J%' AND a.last_name LIKE 'D%'
    -> UNION ALL
    -> SELECT c.first_name, c.last_name
    -> FROM customer c
    -> WHERE c.first_name LIKE 'J%' AND c.last_name LIKE 'D%'
    -> ORDER BY lname, fname;
+----------+-----------+
```

```
| fname    | lname     |
+----------+-----------+
| JENNIFER | DAVIS     |
| JENNIFER | DAVIS     |
| JUDY     | DEAN      |
| JODIE    | DEGENERES |
| JULIANNE | DENCH     |
+----------+-----------+
5 rows in set (0.00 sec)
```

The column names specified in the two queries are different in this example. If you specify a column name from the second query in your `order by` clause, you will see the following error:

```
mysql> SELECT a.first_name fname, a.last_name lname
    -> FROM actor a
    -> WHERE a.first_name LIKE 'J%' AND a.last_name LIKE 'D%'
    -> UNION ALL
    -> SELECT c.first_name, c.last_name
    -> FROM customer c
    -> WHERE c.first_name LIKE 'J%' AND c.last_name LIKE 'D%'
    -> ORDER BY last_name, first_name;
ERROR 1054 (42S22): Unknown column 'last_name' in 'order clause'
```

I recommend giving the columns in both queries identical column aliases in order to avoid this issue.

Set Operation Precedence

If your compound query contains more than two queries using different set operators, you need to think about the order in which to place the queries in your compound statement to achieve the desired results. Consider the following three-query compound statement:

```
mysql> SELECT a.first_name, a.last_name
    -> FROM actor a
    -> WHERE a.first_name LIKE 'J%' AND a.last_name LIKE 'D%'
    -> UNION ALL
    -> SELECT a.first_name, a.last_name
    -> FROM actor a
    -> WHERE a.first_name LIKE 'M%' AND a.last_name LIKE 'T%'
```

```
    -> UNION
    -> SELECT c.first_name, c.last_name
    -> FROM customer c
    -> WHERE c.first_name LIKE 'J%' AND c.last_name LIKE 'D%';
+------------+-----------+
| first_name | last_name |
+------------+-----------+
| JENNIFER   | DAVIS     |
| JUDY       | DEAN      |
| JODIE      | DEGENERES |
| JULIANNE   | DENCH     |
| MARY       | TANDY     |
| MENA       | TEMPLE    |
+------------+-----------+
6 rows in set (0.00 sec)
```

This compound query includes three queries that return sets of nonunique names; the first and second queries are separated with the `union all` operator, while the second and third queries are separated with the `union` operator. While it might not seem to make much difference where the `union` and `union all` operators are placed, it does, in fact, make a difference. Here's the same compound query with the set operators reversed:

```
mysql> SELECT a.first_name, a.last_name
    -> FROM actor a
    -> WHERE a.first_name LIKE 'J%' AND a.last_name LIKE 'D%'
    -> UNION
    -> SELECT a.first_name, a.last_name
    -> FROM actor a
    -> WHERE a.first_name LIKE 'M%' AND a.last_name LIKE 'T%'
    -> UNION ALL
    -> SELECT c.first_name, c.last_name
    -> FROM customer c
    -> WHERE c.first_name LIKE 'J%' AND c.last_name LIKE 'D%';
+------------+-----------+
| first_name | last_name |
+------------+-----------+
| JENNIFER   | DAVIS     |
| JUDY       | DEAN      |
| JODIE      | DEGENERES |
| JULIANNE   | DENCH     |
| MARY       | TANDY     |
| MENA       | TEMPLE    |
| JENNIFER   | DAVIS     |
```

```
+-----------+-----------+
7 rows in set (0.00 sec)
```

Looking at the results, it's obvious that it *does* make a difference how the compound query is arranged when using different set operators. In general, compound queries containing three or more queries are evaluated in order from top to bottom, but with the following caveats:

- The ANSI SQL specification calls for the `intersect` operator to have precedence over the other set operators.
- You may dictate the order in which queries are combined by enclosing multiple queries in parentheses.

MySQL does not yet allow parentheses in compound queries, but if you are using a different database server, you can wrap adjoining queries in parentheses to override the default top-to-bottom processing of compound queries, as in:

```
SELECT a.first_name, a.last_name
FROM actor a
WHERE a.first_name LIKE 'J%' AND a.last_name LIKE 'D%'
UNION
(SELECT a.first_name, a.last_name
 FROM actor a
 WHERE a.first_name LIKE 'M%' AND a.last_name LIKE 'T%'
 UNION ALL
 SELECT c.first_name, c.last_name
 FROM customer c
 WHERE c.first_name LIKE 'J%' AND c.last_name LIKE 'D%'
)
```

For this compound query, the second and third queries would be combined using the `union all` operator, then the results would be combined with the first query using the `union` operator.

Test Your Knowledge

The following exercises are designed to test your understanding of set operations. See Appendix B for the answers to these exercises.

Exercise 6-1

If set A = {L M N O P} and set B = {P Q R S T}, what sets are generated by the following operations?

- `A union B`
- `A union all B`
- `A intersect B`
- `A except B`

Exercise 6-2

Write a compound query that finds the first and last names of all actors and customers whose last name starts with L.

Exercise 6-3

Sort the results from Exercise 6-2 by the `last_name` column.

Previous chapter

‹ 5. Querying Multiple Tables

Next chapter

7. Data Generation, Manipulation, and Conversion ›